

Homework3_DSBD

Ingegneria Informatica LM-32

Anno: 2025-2026

Giuseppe Ravesi

Sommario

1	Introduzione	4
2	Architettura del Sistema.....	5
2.1	Visione Generale	5
2.2	Microservizi Applicativi	5
2.3	Persistenza dati.....	6
2.4	Comunicazioni tra componenti	6
2.5	Ingress Controller.....	7
2.6	Monitoring e osservabilità	7
3	Comunicazioni e Flussi di Interazione.....	8
3.1	Interazioni con il mondo esterno.....	8
3.2	Flussi Sincroni (http e gRPC)	9
3.2.1	Client → UserManager (http)	9
3.2.2	Client → DataCollector (http).....	9
3.2.3	DataCollector → UserManager (http).....	9
3.3	Flussi Asincroni (Kafka)	10
3.3.1	DataCollector → Alert System	10
3.3.2	Alert System → Alert Notifier	10
3.3.3	Alert Notifier → SMTP Server	10
3.4	Gestione degli Errori e della Resilienza.....	11
3.4.1		12
4	Deployment su Kubernetes	13
4.1	Configurazione delle Risorse Kubernetes.....	13
4.1.1	Deployment e Scaling dei Servizi Stateless.....	13
4.1.2	Deployment dei Servizi Statefull con Vincoli Applicativi.....	14

4.1.3	Gestione dei Workload Stateful Persistenti	14
4.1.4	Networking.....	15
4.2	Gestione Operativa, Sicurezza e Ottimizzazione delle Risorse	15
4.2.1	Gestione della Configurazione e dei Secret	15
4.2.2	Affidabilità e Self-Healing.....	15
5	Osservabilità del Sistema	16
5.1	Architettura del Sistema di Monitoraggio.....	16
5.2	Metriche implementate	16
5.2.1	Metriche nel Data Collector	16
5.2.2	Metriche dell'Alert System.....	17
5.2.3	Metriche dell'User Manager	18

1 Introduzione

Il presente documento descrive il progetto sviluppato per **Homework 3** del corso *Sistemi Distribuiti e Big Data*, che rappresenta un'evoluzione naturale dei sistemi progettati e implementati nei precedenti Homework 1 e Homework 2.

L'obiettivo principale di Homework 3 è stato quello di **migrare l'architettura a microservizi precedentemente realizzata con Docker Compose verso un ambiente orchestrato basato su Kubernetes**, introducendo al contempo strumenti e meccanismi tipici di contesti di produzione reali, quali:

- orchestrazione dei container;
- gestione dichiarativa delle risorse;
- service discovery e networking interno;
- osservabilità tramite metriche;
- ingress controller come punto di accesso unico.

L'applicazione mantiene invariata la **logica di business** sviluppata negli homework precedenti, continuando a fornire funzionalità di:

- gestione utenti;
- raccolta dati di volo tramite OpenSky Network;
- comunicazione asincrona basata su Apache Kafka;
- sistema di alert con soglie configurabili;
- invio di notifiche tramite SMTP.

Tuttavia, l'intero sistema è stato **ripensato dal punto di vista infrastrutturale**, sostituendo il modello di deployment basato su Docker Compose con una **architettura completamente Kubernetes-native**.

In particolare, Homework 3 introduce:

- l'utilizzo di **Kind (Kubernetes in Docker)** per la creazione di un cluster locale;
- l'uso di **Deployment e StatefulSet** per la gestione dei microservizi e dei database;
- l'adozione di **Service (ClusterIP e Headless)** per l'esposizione e la discovery dei servizi;
- l'integrazione di **NGINX Ingress Controller** come API Gateway del sistema;
- l'introduzione di **Prometheus** per la raccolta e visualizzazione delle metriche applicative;
- la separazione esplicita tra configurazioni e segreti tramite **ConfigMap e Secret**.

L'obiettivo didattico del progetto non è stato esclusivamente quello di “far funzionare” l'applicazione in Kubernetes, ma anche di:

- comprendere le differenze concettuali tra orchestrazione e containerizzazione semplice;
- motivare le scelte architetturali adottate;
- analizzare alternative progettuali possibili e spiegare perché non sono state perseguite;
- costruire una soluzione coerente, modulare e facilmente estendibile.

Nel seguito del documento verranno descritti:

- l'architettura complessiva del sistema e i microservizi coinvolti;
- le modalità di comunicazione sincrona e asincrona;

- le scelte effettuate per il deployment su Kubernetes;
- i meccanismi di osservabilità introdotti;
- le procedure di build, deploy e test del sistema.

[Placeholder – Diagramma Architetture Complessivo]

Diagramma che rappresenta tutti i microservizi, Kafka, database, Prometheus e Ingress Controller all'interno del cluster Kubernetes.

2 Architettura del Sistema

L'architettura del sistema sviluppato per Homework 3 è basata su un insieme di microservizi containerizzati e orchestrati tramite Kubernetes, progettata per garantire modularità, isolamento, scalabilità e osservabilità. Il sistema adotta un'architettura a microservizi eterogenei, in cui ciascun componente è responsabile di una specifica funzionalità applicativa e comunica con gli altri servizi tramite protocolli ben definiti (HTTP, gRPC e Kafka).

2.1 Visione Generale

Dal punto di vista logico, il sistema è suddiviso nei seguenti livelli principali:

- **Application Layer**
Contiene i microservizi applicativi che implementano la logica di business.
- **Stateful Layer**
Comprende i database e i servizi che richiedono persistenza dello stato.
- **Messaging Layer**
Basato su Apache Kafka per la comunicazione asincrona event-driven.
- **Ingress & Networking Layer**
Gestisce l'accesso dall'esterno e il routing delle richieste.
- **Monitoring Layer**
Responsabile della raccolta e visualizzazione delle metriche.

Tutti i componenti sono eseguiti all'interno di un **cluster Kubernetes locale**, creato mediante **Kind**, che consente di simulare un ambiente di orchestrazione reale mantenendo un contesto di sviluppo controllato.

2.2 Microservizi Applicativi

L'Application Layer è composto dai seguenti microservizi:

User Manager

- Gestisce la registrazione e il recupero delle informazioni degli utenti.
- Espone API REST per la gestione degli utenti.
- Espone un servizio **gRPC** utilizzato dal Data Collector per verificare l'esistenza di un utente.
- È stateless e viene deployato tramite **Deployment**.

Data Collector

- Gestisce gli interessi aeroportuali associati agli utenti.
- Effettua chiamate all'API OpenSky Network per la raccolta dei dati di volo.
- Implementa un **Circuit Breaker** per proteggere le chiamate verso OpenSky.
- Produce eventi su Kafka al termine di ogni raccolta.
- Espone API REST per l'interazione esterna.
- È stateless e viene deployato tramite **Deployment**.

Alert System

- Consuma eventi dal topic Kafka `to-alert-system`.
- Recupera le soglie configurate dal database.
- Valuta le condizioni di alert (superamento soglia alta o inferiore).
- Produce eventi su Kafka verso il topic `to-notifier`.
- Non è esposto all'esterno.
- È stateless e viene deployato tramite **Deployment**.

Alert Notifier

- Consuma eventi dal topic Kafka `to-notifier`.
- Invia notifiche e-mail tramite protocollo SMTP.
- Utilizza credenziali esterne gestite tramite Secret Kubernetes.
- Non espone API HTTP.
- È stateless e viene deployato tramite **Deployment**.

2.3 Persistenza dati

Il sistema utilizza il pattern **Database-per-Service**, mantenendo una separazione netta tra i dati:

- **User DB**
Database PostgreSQL contenente le informazioni sugli utenti.
- **Data DB**
Database PostgreSQL contenente gli interessi aeroportuali e le soglie di alert.

Entrambi i database:

- sono deployati tramite **StatefulSet**;
- utilizzano **PersistentVolumeClaim** per garantire la persistenza dei dati;
- sono esposti tramite **Service Headless**, per consentire un indirizzamento stabile dei pod.

2.4 Comunicazioni tra componenti

Il sistema utilizza più modalità di comunicazione:

- **HTTP REST**
Utilizzato per l'accesso esterno e per la comunicazione sincrona tra client e microservizi.
- **gRPC**
Utilizzato per la comunicazione interna sincrona tra Data Collector e User Manager.

- **Kafka (pub/sub)**

Utilizzato per la comunicazione asincrona tra Data Collector, Alert System e Alert Notifier.

Questa combinazione consente di:

- mantenere bassa la dipendenza temporale tra i servizi;
- isolare i fallimenti;
- migliorare la stabilità del sistema

2.5 Ingress Controller

L'accesso dall'esterno è gestito tramite **NGINX Ingress Controller**, che funge da **API Gateway** del sistema.

L'Ingress:

- espone un unico endpoint (`http://localhost`);
- instrada le richieste verso:
 - `/users/*` → User Manager
 - `/collector/*` → Data Collector;
- permette di effettuare health check centralizzati;
- isola completamente i servizi interni dal mondo esterno.

2.6 Monitoring e osservabilità

Per il monitoraggio del sistema è stato integrato **Prometheus**, che raccoglie metriche da:

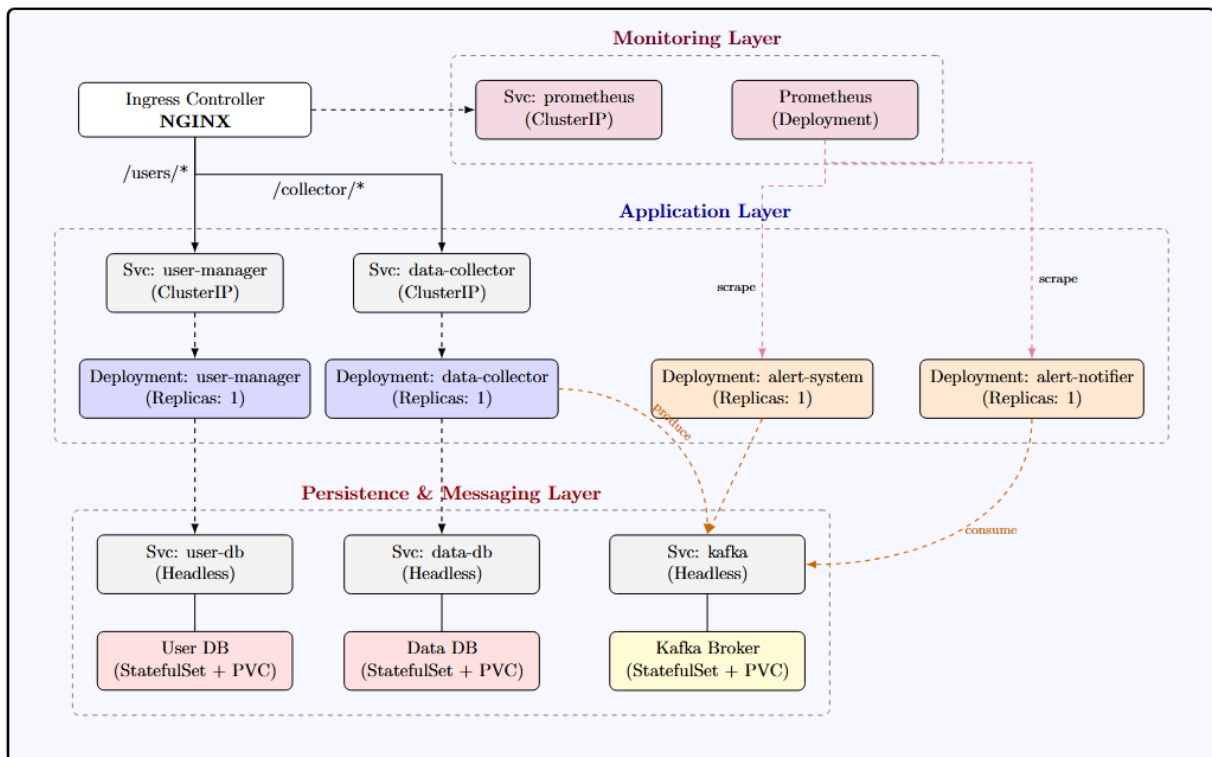
- User Manager
- Data Collector
- Alert System
- Alert Notifier

Ogni microservizio espone un endpoint `/metrics` conforme allo standard Prometheus.

Le metriche includono:

- metriche di runtime Python;
- metriche HTTP (request count, latency);
- metriche applicative personalizzate (raccolte, errori, eventi Kafka).

Kubernetes Cluster (Kind)



3 Comunicazioni e Flussi di Interazione

Questo capitolo descrive nel dettaglio **come i microservizi comunicano tra loro e come il sistema interagisce con il mondo esterno**, evidenziando i flussi principali e le motivazioni progettuali che hanno portato alla scelta dei diversi meccanismi di comunicazione.

Il sistema utilizza una **combinazione di comunicazioni sincrone e asincrone**, al fine di bilanciare semplicità, affidabilità e scalabilità.

3.1 Interazioni con il mondo esterno

Tutte le interazioni esterne avvengono tramite **NGINX Ingress Controller**, che funge da **API Gateway**.

Il client (es. Postman o browser):

- invia richieste HTTP a `http://localhost`;
- non ha conoscenza delle porte o dei servizi interni;
- interagisce esclusivamente con endpoint REST pubblici.

L'Ingress si occupa di:

- terminare la connessione HTTP;
- instradare la richiesta verso il microservizio corretto;
- nascondere completamente l'architettura interna.

Questo approccio consente di:

- semplificare l'uso del sistema;
- migliorare la sicurezza;
- facilitare eventuali estensioni future (rate limiting, autenticazione, TLS).

3.2 Flussi Sincroni (http e gRPC)

3.2.1 Client → UserManager (http)

Il flusso più semplice prevede l'interazione diretta tra il client e il User Manager:

1. Il client invia una richiesta HTTP all'Ingress.
2. L'Ingress inoltra la richiesta al servizio `user-manager`.
3. L'User Manager interagisce con il proprio database.
4. La risposta viene restituita al client.

Questo flusso è completamente **sincrono** e viene utilizzato per:

- registrazione utenti;
- interrogazioni sui dati utente;
- health check del servizio.

3.2.2 Client → DataCollector (http)

Il Data Collector espone API REST per:

- registrare interessi aeroportuali;
- aggiornare soglie di alert;
- avviare manualmente una raccolta dati.

Il flusso è analogo a quello del User Manager:

1. Client → Ingress
2. Ingress → Data Collector
3. Data Collector → Data DB
4. Risposta al client

3.2.3 DataCollector → UserManager (http)

Per verificare l'esistenza di un utente, il Data Collector utilizza **gRPC** invece di HTTP.

Motivazioni della scelta:

- comunicazione interna ad alta frequenza;
- payload ridotto;
- contratto fortemente tipizzato;
- minore overhead rispetto a REST.

Flusso:

1. Data Collector invia una richiesta gRPC al User Manager.
2. L'User Manager verifica l'utente nel proprio database.
3. Restituisce l'esito al Data Collector.

Questo flusso è **sincrono**, ma confinato esclusivamente all'interno del cluster.

3.3 Flussi Asincroni (Kafka)

Per disaccoppiare i microservizi e ridurre la dipendenza temporale, il sistema utilizza **Apache Kafka** come message broker.

3.3.1 DataCollector → Alert System

Al termine di una raccolta dati (manuale o schedulata), il Data Collector:

- produce un messaggio sul topic Kafka `to-alert-system`.

Il messaggio contiene:

- e-mail dell'utente;
- aeroporto;
- numero di arrivi e partenze;
- timestamp.

Questo approccio consente al Data Collector di:

- completare rapidamente la richiesta;
- non dipendere dallo stato del sistema di alert;
- evitare blocchi o ritardi.

3.3.2 Alert System → Alert Notifier

L'Alert System:

- consuma messaggi dal topic `to-alert-system`;
- recupera le soglie dal database;
- valuta le condizioni di alert.

Se una soglia è violata:

- produce un messaggio sul topic `to-notifier`.

Il messaggio contiene:

- e-mail destinataria;
- aeroporto;
- descrizione della condizione di alert.

3.3.3 Alert Notifier → SMTP Server

L'Alert Notifier:

- consuma eventi dal topic `to-notifier`;
- invia e-mail tramite un server SMTP esterno (es. Gmail).

Questo è l'unico punto in cui il sistema interagisce direttamente con un servizio esterno non HTTP.

L'utilizzo di Kafka permette di:

- isolare completamente l'invio email;
- evitare che problemi SMTP impattino gli altri servizi;
- migliorare la resilienza complessiva.

3.4 Gestione degli Errori e della Resilienza

Sono state adottate diverse strategie per aumentare la robustezza del sistema:

- **Circuit Breaker** nel Data Collector per proteggere le chiamate verso OpenSky Network.
- **Retry automatici** gestiti da Kafka per i consumer.
- **Isolamento dei fallimenti** grazie alla comunicazione asincrona.
- **Health check HTTP** per consentire a Kubernetes di rilevare servizi non funzionanti.

Kubernetes Cluster (Kind)

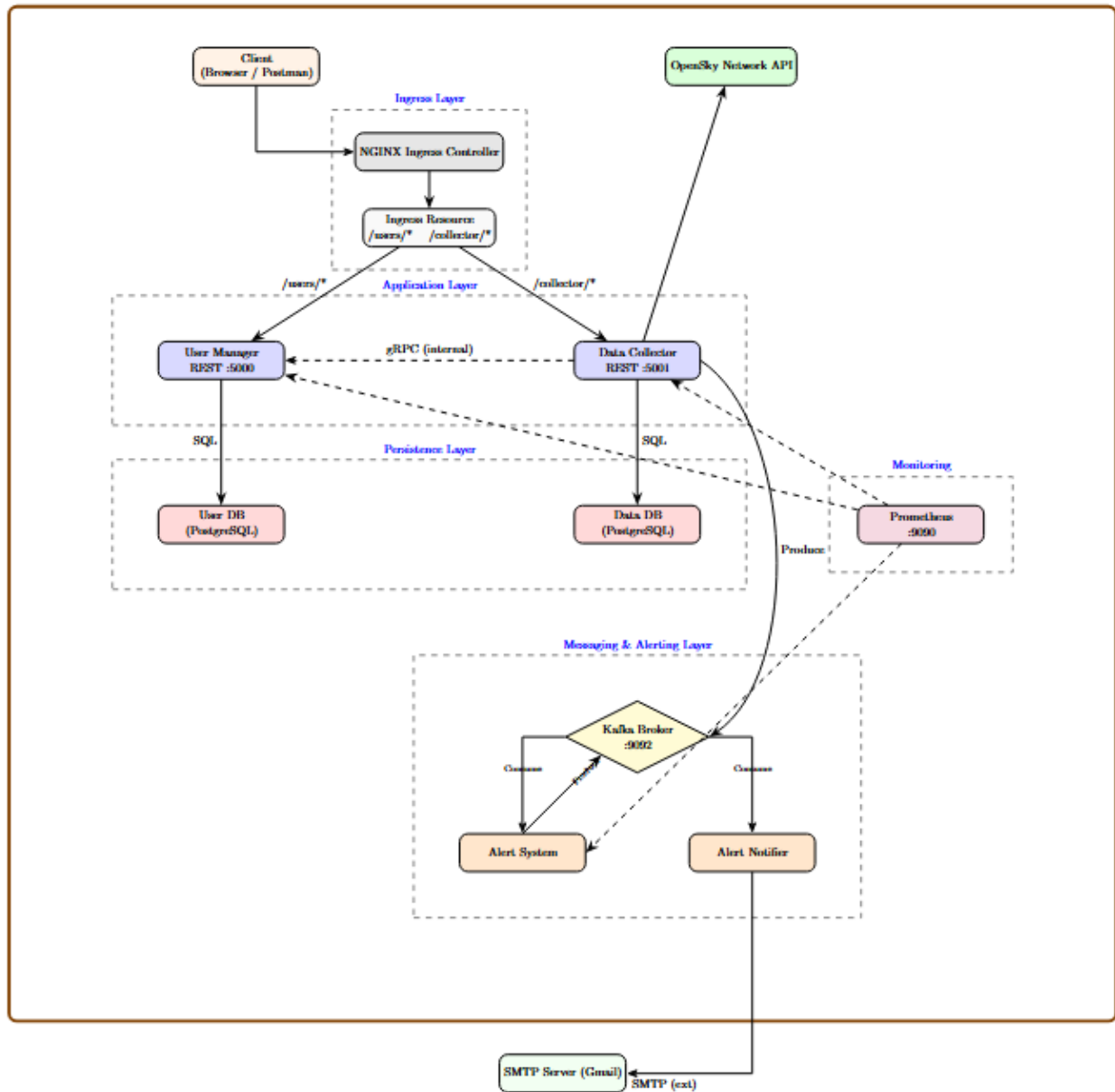


Figura 3.1 Diagramma interazioni

4 Deployment su Kubernetes

Con riferimento all'architettura descritta nei capitoli precedenti, l'infrastruttura del sistema è stata realizzata adottando il paradigma *Infrastructure as Code* (IaC), mediante la definizione di manifesti dichiarativi in formato YAML per Kubernetes.

L'obiettivo principale del deployment è stato quello di riprodurre un ambiente distribuito realistico, garantendo isolamento dei componenti, gestione automatica del ciclo di vita dei servizi, osservabilità e resilienza operativa, pur mantenendo una configurazione compatibile con un ambiente di sviluppo locale.

Il cluster Kubernetes è stato realizzato tramite **Kind (Kubernetes in Docker)** ed eseguito su **Docker Desktop**, configurato come cluster single-node. Sebbene tale configurazione non rappresenti un ambiente di produzione, essa risulta adeguata a validare il comportamento dei microservizi, le interazioni asincrone tramite Kafka, i meccanismi di autoscaling logico e le politiche di gestione dello stato.

4.1 Configurazione delle Risorse Kubernetes

La definizione delle risorse Kubernetes è stata modellata in funzione delle caratteristiche architetturali dei singoli microservizi, distinguendo chiaramente tra **componenti stateless** e **componenti stateful**, e adottando pattern di deployment differenti in base ai vincoli di stato, persistenza e concorrenza.

4.1.1 Deployment e Scaling dei Servizi Stateless

I microservizi **User Manager**, **Alert System** e **Alert Notifier** sono stati progettati come servizi **stateless**, in quanto non mantengono stato applicativo locale persistente e delegano completamente la persistenza a sistemi esterni (database PostgreSQL o Kafka).

Per questi servizi è stato adottato il controller **Deployment**, che consente:

- la gestione automatica del ciclo di vita dei Pod;
- il riavvio automatico in caso di crash (*self-healing*);
- la possibilità di scalare orizzontalmente il numero di repliche.

Nel caso dello **User Manager**, nonostante il servizio esponga API HTTP e gRPC, lo stato applicativo rimane interamente nel database utenti. Di conseguenza, l'aumento del numero di repliche non introduce problemi di consistenza, poiché tutte le istanze accedono alla stessa base dati.

Per i servizi **Alert System** e **Alert Notifier**, il modello di concorrenza non è governato direttamente da Kubernetes, bensì dal broker Kafka. In particolare:

- ogni istanza del servizio si comporta come un *Kafka Consumer*;
- il parallelismo effettivo è determinato dal numero di partizioni del topic;

- Kafka garantisce che una singola partizione venga consumata da una sola istanza per gruppo di consumer.

Questo approccio consente di scalare il carico in modo controllato, evitando duplicazioni nella gestione dei messaggi senza richiedere logica di coordinamento applicativo.

4.1.2 Deployment dei Servizi Statefull con Vincoli Applicativi

Il **Data Collector** rappresenta un caso particolare dal punto di vista architetturale. Sebbene interagisca con un database relazionale per la persistenza dei dati, il servizio mantiene anche **stato volatile in memoria**, legato alla gestione dello scheduling delle operazioni di raccolta (job pianificati e chiamate verso OpenSky Network).

A causa di questi vincoli:

- una replica multipla del Data Collector comporterebbe la duplicazione dei job pianificati;
- ciò potrebbe generare chiamate ridondanti verso OpenSky, violando i limiti di utilizzo dell'API esterna;
- si rischierebbero inoltre condizioni di race sulla scrittura dei dati raccolti.

Per tali motivi, il Data Collector è stato configurato come **singleton applicativo**, con un numero di repliche fissato a uno (`replicas: 1`).

La resilienza del servizio non è ottenuta tramite replica attiva, ma attraverso il meccanismo di **self-healing di Kubernetes**, che riavvia automaticamente il Pod in caso di errore o crash.

Una possibile evoluzione futura dell'architettura potrebbe prevedere:

- la separazione del componente di scheduling in un microservizio dedicato;
- oppure l'utilizzo di Kafka come meccanismo di coordinamento (leader election su consumer group);
- consentendo una replica controllata del Data Collector senza duplicare l'esecuzione dei job.

Tali soluzioni, tuttavia, non sono state adottate per mantenere la complessità architetturale proporzionata agli obiettivi del progetto.

4.1.3 Gestione dei Workload Stateful Persistenti

I componenti che richiedono persistenza stabile e identità di rete consistente sono:

- **PostgreSQL User DB**
- **PostgreSQL Data DB**
- **Kafka Broker**

Per questi servizi è stato utilizzato il controller **StatefulSet**, che garantisce:

- nomi di Pod stabili e deterministici (es. `kafka-0`, `user-db-0`);
- associazione persistente tra Pod e volume di storage;
- ricostruzione automatica del Pod mantenendo i dati in caso di riavvio o ripianificazione.

La persistenza dei dati è assicurata tramite **PersistentVolumeClaim (PVC)**, montati all'interno dei container. In questo modo, anche in caso di riavvio del cluster o del singolo Pod, i dati rimangono disponibili.

Nel caso di Kafka, l'uso di StatefulSet è essenziale per garantire:

- stabilità dell'identità del broker;

- corretta risoluzione DNS dei Pod;
- funzionamento corretto delle logiche di leader election e replica (anche in configurazione single-node).

4.1.4 Networking

La comunicazione tra i microservizi all'interno del cluster è governata dall'uso di differenti tipologie di **Service**, selezionate in base alle esigenze di indirizzamento e accesso.

- **ClusterIP**
Utilizzato per i servizi stateless (User Manager, Data Collector, Prometheus), fornisce un IP virtuale stabile e bilanciamento del carico tra le repliche.
- **Headless Service (clusterIP: None)**
Utilizzato per Kafka e i database, consente la risoluzione DNS diretta degli IP dei singoli Pod. Questa configurazione è fondamentale per i sistemi distribuiti che necessitano di identificare specifiche istanze.
- **Assenza di Service per Consumer puri**
I servizi Alert System e Alert Notifier non espongono endpoint HTTP verso l'interno del cluster e interagiscono esclusivamente tramite Kafka. Per questi componenti non è stato definito alcun Service, riducendo la superficie di esposizione e rispettando il principio del *least privilege*.

L'accesso dall'esterno al cluster è gestito tramite **NGINX Ingress Controller**, che funge da API Gateway e punto di ingresso unico verso il sistema.

4.2 Gestione Operativa, Sicurezza e Ottimizzazione delle Risorse

4.2.1 Gestione della Configurazione e dei Secret

Per garantire portabilità, sicurezza e separazione delle responsabilità, la configurazione applicativa è stata esternalizzata rispetto al codice sorgente.

- I parametri non sensibili (endpoint, nomi dei topic Kafka, porte, hostname dei servizi) sono gestiti tramite **ConfigMap**.
- Le informazioni sensibili (password dei database, credenziali SMTP, token API) sono gestite tramite **Secret** Kubernetes.

Entrambe le risorse vengono iniettate nei Pod sotto forma di **variabili d'ambiente**, evitando l'hardcoding delle configurazioni nel codice e consentendo una facile riconfigurazione tra ambienti diversi.

4.2.2 Affidabilità e Self-Healing

La resilienza dei servizi è garantita tramite l'uso combinato di:

- **Liveness Probe**, per individuare container bloccati o non funzionanti;
- **Readiness Probe**, per evitare l'invio di traffico a Pod non pronti;
- **Startup Probe**, per gestire correttamente fasi di inizializzazione lente (in particolare per Kafka).

Questi meccanismi permettono a Kubernetes di:

- riavviare automaticamente i container in errore;
- escludere temporaneamente i Pod non pronti dal routing;

- evitare riavvii prematuri durante l'avvio dei servizi stateful.

5 Osservabilità del Sistema

In un sistema distribuito basato su microservizi, l'osservabilità rappresenta un requisito fondamentale per garantire affidabilità, individuare anomalie e comprendere il comportamento globale dell'applicazione. Nel contesto del progetto DSBD, il monitoraggio è stato progettato per fornire una visione chiara sia dello stato dei singoli microservizi sia delle interazioni asincrone mediate dal broker Kafka.

A tal fine, è stato adottato **Prometheus** come sistema di monitoraggio centralizzato, integrato nativamente con Kubernetes e configurato per raccogliere metriche applicative personalizzate esposte dai microservizi.

5.1 Architettura del Sistema di Monitoraggio

Il sistema di osservabilità è basato su un'architettura *pull-based*, in linea con il modello di Prometheus. In particolare:

- ogni microservizio espone un endpoint HTTP `/metrics`;
- Prometheus interroga periodicamente tali endpoint;
- le metriche raccolte vengono memorizzate nel *time-series database* interno di Prometheus;
- l'interfaccia web di Prometheus consente interrogazioni e visualizzazioni in tempo reale.

Prometheus è stato deployato all'interno del cluster Kubernetes come servizio stateless, ed è accessibile tramite Ingress, consentendo l'analisi delle metriche senza esporre direttamente le singole istanze dei microservizi.

5.2 Metriche implementate

5.2.1 Metriche nel Data Collector

Il Data Collector espone metriche che consentono di osservare:

- il numero totale di richieste HTTP ricevute;
- la durata dell'ultima raccolta di dati;
- il numero di record salvati per ciascuna esecuzione;
- il numero di chiamate effettuate verso OpenSky Network;
- eventuali fallimenti nella fase di raccolta.

In particolare:

Metriche http:

```
□ http_requests_total
```

Counter – Numero totale di richieste HTTP gestite

Label principali:

- `service="data_collector"`
- `node`
- `method`
- `path`
- `status`

□ `http_requests_created`

Gauge – Timestamp di creazione delle richieste HTTP (per serie temporali)

□ `http_last_request_seconds`

Gauge – Tempo di esecuzione dell'ultima richiesta HTTP

Label:

- `service`
- `node`
- `method`
- `path`

Metriche di Collection:

□ `collection_runs_total`

Counter – Numero totale di esecuzioni di raccolta

Label:

- `service="data_collector"`
- `node`
- `trigger (manual / scheduler)`

□ `collection_runs_created`

Gauge – Timestamp di creazione dell'ultima run

□ `collection_failures_total`

Counter – Numero totale di fallimenti della raccolta

□ `collection_last_run_seconds`

Gauge – Durata dell'ultima raccolta completata

□ `collection_records_saved_last`

Gauge – Numero di record salvati nell'ultima run

Metriche OpenSky:

□ `opensky_calls_total`

Counter – Numero totale di chiamate verso OpenSky

Label:

- `service`
- `node`
- `result (ok / error)`

□ `opensky_calls_created`

Gauge – Timestamp dell'ultima chiamata OpenSky

□ `opensky_last_call_seconds`

Gauge – Tempo di esecuzione dell'ultima chiamata OpenSky

5.2.2 Metriche dell'Alert System

L'Alert System, essendo un consumer Kafka, espone metriche orientate al flusso di messaggi e alla logica di business. In particolare:

- numero totale di messaggi consumati da Kafka;
- numero di messaggi elaborati con successo;

- numero di messaggi scartati o non validi;
- tempo di elaborazione dell'ultimo messaggio;
- numero di query effettuate sul database per il recupero delle soglie;
- numero totale di alert generati.

Metriche Kafka (Consumer / Producer):

□ kafka_messages_consumed_total

Counter – Numero totale di messaggi Kafka consumati

Label:

- service="alert_system"
- node
- topic="to-alert-system"
- result (ok / empty)

□ kafka_messages_consumed_created

Gauge – Timestamp di creazione dell'ultima metrica di consumo

□ kafka_last_message_processing_seconds

Gauge – Tempo di processamento dell'ultimo messaggio Kafka

Metriche Database:

□ db_threshold_queries_total

Counter – Numero totale di query al database per recupero soglie

Label:

- service="alert_system"
- node
- result (ok / error)

□ db_threshold_queries_created

Gauge – Timestamp dell'ultima query di soglia

Metriche Alert:

□ alerts_sent_total

Counter – Numero totale di alert generati e inviati verso il notifier

Label:

- service="alert_system"
- node
- topic="to-notifier"
- result (ok / error)

□ alerts_sent_created

Gauge – Timestamp dell'ultimo alert inviato

5.2.3 Metriche dell'User Manager

Il microservizio **User Manager** espone un endpoint `/metrics` conforme allo standard Prometheus, integrato direttamente nell'applicazione Flask. Le metriche sono state progettate per monitorare principalmente due aspetti: il comportamento del layer HTTP e la disponibilità dell'accesso al database.

In particolare, il servizio espone:

- **Metriche HTTP**

- o `http_requests_total` (Counter)

Conta il numero totale di richieste HTTP gestite, etichettate per:

- `service` (nome del servizio)
 - `node` (nodo Kubernetes su cui gira il Pod)
 - `path`
 - `method`
 - `status`

Questa metrica permette di verificare frequenza e tipologia delle chiamate, nonché di distinguere risposte corrette da errori applicativi.

- o `http_last_request_seconds` (Gauge)

Misura il tempo di esecuzione dell'ultima richiesta gestita, etichettata per:

- `service`
 - `node`
 - `path`
 - `method`

Tale metrica è utile per identificare degradazioni prestazionali e pattern anomali (es. latenza elevata per la registrazione).

- **Metriche di runtime Python**

Prometheus espone inoltre metriche standard del processo (memoria, CPU, garbage collector) che consentono di analizzare il comportamento del servizio in termini di risorse.

Dal punto di vista interpretativo, l'osservazione combinata di `http_requests_total` e `http_last_request_seconds` consente di:

- correlare il carico sul servizio con la latenza osservata;
- valutare la stabilità in presenza di richieste ripetute (es. test di idempotenza tramite `request_id`);
- verificare indirettamente il corretto funzionamento del service discovery e dell'Ingress (attraverso l'incremento delle richieste su endpoint `/health`, `/register`, ecc.).